EXPERIMENT

Build an Artificial Neural Network by implementing the Backpropagation algorithm and test the same using appropriate data sets.

Name : S.Nagulan

Reg No : 1924260216

(This experiment is divided into 5 problems)

Problem 1: XOR Gate Data Set

Aim

To build an Artificial Neural Network (2-4-1 architecture) by implementing the Backpropagation algorithm in Python and to test it using the XOR Gate Data Set, which represents the XOR logic gate, a problem that is not linearly separable.

Procedure

1. Store the input data and the expected output of the XOR Gate Data Set; normalise the values to the range 0 - 1 where required.
2. Fix the network structure (2 input, 4 hidden and 1 output neurons), the learning rate (0.5) and the number of epochs (10000); initialise the weights randomly and the biases with zero.
3. Forward propagation: calculate the hidden layer and output layer activations using the sigmoid function.
4. Calculate the error (expected output - actual output) and propagate it backwards to find the delta (error gradient) of the output layer and of the hidden layer.
5. Update all weights and biases using the delta rule: weight = weight + learning rate x input x delta.
6. Repeat steps 3 to 5 for all epochs while printing the loss (MSE), then test the trained network and display the predicted output.

Coding (Python)

```
# Backpropagation Neural Network - XOR Gate Data Set
import numpy as np

np.random.seed(1)

# Input data and expected output
X = np.array([
    [0, 0],
    [0, 1],
    [1, 0],
    [1, 1],
])
y = np.array([
    [0],
    [1],
    [1],
    [0],
])

def sigmoid(x):
    return 1 / (1 + np.exp(-x))

def sigmoid_derivative(a):              # a = sigmoid(x)
    return a * (1 - a)

# Network structure: 2 input - 4 hidden - 1 output neurons
input_n, hidden_n, output_n = 2, 4, 1
```

```
learning_rate = 0.5
epochs = 10000

# Step 1: initialise weights randomly and biases with zero
w_hidden = np.random.uniform(-1, 1, (input_n, hidden_n))
b_hidden = np.zeros((1, hidden_n))
w_output = np.random.uniform(-1, 1, (hidden_n, output_n))
b_output = np.zeros((1, output_n))

for epoch in range(1, epochs + 1):
    # Step 2: forward propagation
    hidden = sigmoid(np.dot(X, w_hidden) + b_hidden)
    output = sigmoid(np.dot(hidden, w_output) + b_output)

    # Step 3: calculate the error
    error = y - output

    # Step 4: backward propagation (delta rule)
    d_output = error * sigmoid_derivative(output)
    d_hidden = np.dot(d_output, w_output.T) * sigmoid_derivative(hidden)

    # Step 5: update weights and biases
    w_output += learning_rate * np.dot(hidden.T, d_output)
    b_output += learning_rate * np.sum(d_output, axis=0, keepdims=True)
    w_hidden += learning_rate * np.dot(X.T, d_hidden)
    b_hidden += learning_rate * np.sum(d_hidden, axis=0, keepdims=True)

    if epoch == 1 or epoch % 2000 == 0:
        print("Epoch", epoch, "  Loss (MSE) =", round(float(np.mean(error ** 2)), 6))

# Step 6: test the trained network
hidden = sigmoid(np.dot(X, w_hidden) + b_hidden)
output = sigmoid(np.dot(hidden, w_output) + b_output)
predicted = (output > 0.5).astype(int)
print("\nTesting the trained network:")
print("Input  | Actual | Output (raw)  | Predicted")
for i in range(len(X)):
    print(X[i].tolist(), "|", y[i].tolist(), "|", np.round(output[i], 3).tolist(), "|", predicted[i].tolist())
accuracy = np.mean(predicted == y) * 100
print("\nFinal Loss (MSE) :", round(float(np.mean((y - output) ** 2)), 6))
print("Accuracy         :", accuracy, "%")
```

Input

Data set (4 examples, 2 input neuron(s), 1 output neuron(s)):

A	B	A XOR B
0	0	0
0	1	1
1	0	1
1	1	0

Output

```
Epoch 1   Loss (MSE) = 0.249726
Epoch 2000   Loss (MSE) = 0.002769
Epoch 4000   Loss (MSE) = 0.000892
Epoch 6000   Loss (MSE) = 0.000516
Epoch 8000   Loss (MSE) = 0.000359
Epoch 10000   Loss (MSE) = 0.000275

Testing the trained network:
Input  | Actual | Output (raw)  | Predicted
[0, 0] | [0] | [0.015] | [0]
[0, 1] | [1] | [0.981] | [1]
[1, 0] | [1] | [0.988] | [1]
[1, 1] | [0] | [0.02] | [0]

Final Loss (MSE) : 0.000275
Accuracy         : 100.0 %
```

Result

Thus the Backpropagation Artificial Neural Network was built and tested successfully on the XOR Gate Data Set. The loss decreased steadily over the epochs to a final MSE of **0.000275** and the network classified the data set with an accuracy of **100.0 %**.

Problem 2: Study Hours and Sleep Hours Data Set

Aim

To build an Artificial Neural Network (2-5-1 architecture) by implementing the Backpropagation algorithm in Python and to test it using the Study Hours and Sleep Hours Data Set, which represents the test score of a student from the hours of study and sleep (regression).

Procedure

1. Store the input data and the expected output of the Study Hours and Sleep Hours Data Set; normalise the values to the range 0 - 1 where required.
2. Fix the network structure (2 input, 5 hidden and 1 output neurons), the learning rate (0.5) and the number of epochs (10000); initialise the weights randomly and the biases with zero.
3. Forward propagation: calculate the hidden layer and output layer activations using the sigmoid function.
4. Calculate the error (expected output - actual output) and propagate it backwards to find the delta (error gradient) of the output layer and of the hidden layer.
5. Update all weights and biases using the delta rule: weight = weight + learning rate x input x delta.
6. Repeat steps 3 to 5 for all epochs while printing the loss (MSE), then test the trained network and display the predicted output.

Coding (Python)

```
# Backpropagation Neural Network - Study Hours and Sleep Hours Data Set
import numpy as np

np.random.seed(1)

# Input data and expected output
X = np.array([
    [2, 9],
    [1, 5],
    [3, 6],
])
y = np.array([
    [92],
    [86],
    [89],
])

# Normalise the data (scale values to the range 0 - 1)
X = X / np.amax(X, axis=0)
y = y / 100

def sigmoid(x):
    return 1 / (1 + np.exp(-x))

def sigmoid_derivative(a):          # a = sigmoid(x)
```

```
    return a * (1 - a)

# Network structure: 2 input - 5 hidden - 1 output neurons
input_n, hidden_n, output_n = 2, 5, 1
learning_rate = 0.5
epochs = 10000

# Step 1: initialise weights randomly and biases with zero
w_hidden = np.random.uniform(-1, 1, (input_n, hidden_n))
b_hidden = np.zeros((1, hidden_n))
w_output = np.random.uniform(-1, 1, (hidden_n, output_n))
b_output = np.zeros((1, output_n))

for epoch in range(1, epochs + 1):
    # Step 2: forward propagation
    hidden = sigmoid(np.dot(X, w_hidden) + b_hidden)
    output = sigmoid(np.dot(hidden, w_output) + b_output)

    # Step 3: calculate the error
    error = y - output

    # Step 4: backward propagation (delta rule)
    d_output = error * sigmoid_derivative(output)
    d_hidden = np.dot(d_output, w_output.T) * sigmoid_derivative(hidden)

    # Step 5: update weights and biases
    w_output += learning_rate * np.dot(hidden.T, d_output)
    b_output += learning_rate * np.sum(d_output, axis=0, keepdims=True)
    w_hidden += learning_rate * np.dot(X.T, d_hidden)
    b_hidden += learning_rate * np.sum(d_hidden, axis=0, keepdims=True)

    if epoch == 1 or epoch % 2000 == 0:
        print("Epoch", epoch, "  Loss (MSE) =", round(float(np.mean(error ** 2)), 6))

# Step 6: test the trained network
hidden = sigmoid(np.dot(X, w_hidden) + b_hidden)
output = sigmoid(np.dot(hidden, w_output) + b_output)
print("\nTesting the trained network:")
print("Normalised inputs :", np.round(X, 3).tolist())
print("Actual score      :", (y * 100).flatten().tolist())
print("Predicted score   :", np.round(output * 100, 2).flatten().tolist())
print("\nFinal Loss (MSE) :", round(float(np.mean((y - output) ** 2)), 6))
```

Input

Data set (3 examples, 2 input neuron(s), 1 output neuron(s)):

Study Hours	Sleep Hours	Test Score
2	9	92
1	5	86
3	6	89

Output

```
Epoch 1   Loss (MSE) = 0.179751
Epoch 2000   Loss (MSE) = 0.000341
Epoch 4000   Loss (MSE) = 0.00022
Epoch 6000   Loss (MSE) = 0.00015
Epoch 8000   Loss (MSE) = 0.000103
Epoch 10000   Loss (MSE) = 6.9e-05

Testing the trained network:
Normalised inputs : [[0.667, 1.0], [0.333, 0.556], [1.0, 0.667]]
Actual score      : [92.0, 86.0, 89.0]
Predicted score   : [90.79, 86.31, 89.71]

Final Loss (MSE) : 6.9e-05
```

Result

Thus the Backpropagation Artificial Neural Network was built and tested successfully on the Study Hours and Sleep Hours Data Set. The loss decreased steadily over the epochs to a final MSE of **6.9e-05** and the predicted scores are close to the actual scores.

Problem 3: Student Pass / Fail Data Set

Aim

To build an Artificial Neural Network (2-4-1 architecture) by implementing the Backpropagation algorithm in Python and to test it using the Student Pass / Fail Data Set, which represents whether a student passes the examination.

Procedure

1. Store the input data and the expected output of the Student Pass / Fail Data Set; normalise the values to the range 0 - 1 where required.
2. Fix the network structure (2 input, 4 hidden and 1 output neurons), the learning rate (0.5) and the number of epochs (5000); initialise the weights randomly and the biases with zero.
3. Forward propagation: calculate the hidden layer and output layer activations using the sigmoid function.
4. Calculate the error (expected output - actual output) and propagate it backwards to find the delta (error gradient) of the output layer and of the hidden layer.
5. Update all weights and biases using the delta rule: weight = weight + learning rate x input x delta.
6. Repeat steps 3 to 5 for all epochs while printing the loss (MSE), then test the trained network and display the predicted output.

Coding (Python)

```
# Backpropagation Neural Network - Student Pass / Fail Data Set
import numpy as np

np.random.seed(1)

# Input data and expected output
X = np.array([
    [1, 40],
    [2, 50],
    [3, 45],
    [4, 60],
    [5, 55],
    [6, 80],
    [7, 75],
    [8, 90],
    [9, 85],
    [10, 95],
])
y = np.array([
    [0],
    [0],
    [0],
    [0],
    [0],
    [1],
    [1],
    [1],
    [1],
```

```
    [1],
])

# Normalise the input features (scale values to the range 0 - 1)
X = X / np.array([10, 100])

def sigmoid(x):
    return 1 / (1 + np.exp(-x))

def sigmoid_derivative(a):          # a = sigmoid(x)
    return a * (1 - a)

# Network structure: 2 input - 4 hidden - 1 output neurons
input_n, hidden_n, output_n = 2, 4, 1
learning_rate = 0.5
epochs = 5000

# Step 1: initialise weights randomly and biases with zero
w_hidden = np.random.uniform(-1, 1, (input_n, hidden_n))
b_hidden = np.zeros((1, hidden_n))
w_output = np.random.uniform(-1, 1, (hidden_n, output_n))
b_output = np.zeros((1, output_n))

for epoch in range(1, epochs + 1):
    # Step 2: forward propagation
    hidden = sigmoid(np.dot(X, w_hidden) + b_hidden)
    output = sigmoid(np.dot(hidden, w_output) + b_output)

    # Step 3: calculate the error
    error = y - output

    # Step 4: backward propagation (delta rule)
    d_output = error * sigmoid_derivative(output)
    d_hidden = np.dot(d_output, w_output.T) * sigmoid_derivative(hidden)

    # Step 5: update weights and biases
    w_output += learning_rate * np.dot(hidden.T, d_output)
    b_output += learning_rate * np.sum(d_output, axis=0, keepdims=True)
    w_hidden += learning_rate * np.dot(X.T, d_hidden)
    b_hidden += learning_rate * np.sum(d_hidden, axis=0, keepdims=True)

    if epoch == 1 or epoch % 1000 == 0:
        print("Epoch", epoch, "  Loss (MSE) =", round(float(np.mean(error ** 2)), 6))

# Step 6: test the trained network
hidden = sigmoid(np.dot(X, w_hidden) + b_hidden)
output = sigmoid(np.dot(hidden, w_output) + b_output)
predicted = (output > 0.5).astype(int)
print("\nTesting the trained network:")
print("Input  | Actual | Output (raw)  | Predicted")
for i in range(len(X)):
    print(X[i].tolist(), "|", y[i].tolist(), "|", np.round(output[i], 3).tolist(), "|", predicted[i].tolist())
accuracy = np.mean(predicted == y) * 100
print("\nFinal Loss (MSE) :", round(float(np.mean((y - output) ** 2)), 6))
print("Accuracy         :", accuracy, "%")
```

Input

Data set (10 examples, 2 input neuron(s), 1 output neuron(s)):

Hours Studied	Attendance %	Result (1 = Pass)
1	40	0

Hours Studied	Attendance %	Result (1 = Pass)
2	50	0
3	45	0
4	60	0
5	55	0
6	80	1
7	75	1
8	90	1
9	85	1
10	95	1

Output

```
Epoch 1   Loss (MSE) = 0.248917
Epoch 1000   Loss (MSE) = 0.00155
Epoch 2000   Loss (MSE) = 0.000575
Epoch 3000   Loss (MSE) = 0.000334
Epoch 4000   Loss (MSE) = 0.00023
Epoch 5000   Loss (MSE) = 0.000173

Testing the trained network:
Input  | Actual | Output (raw)  | Predicted
[0.1, 0.4] | [0] | [0.0] | [0]
[0.2, 0.5] | [0] | [0.0] | [0]
[0.3, 0.45] | [0] | [0.0] | [0]
[0.4, 0.6] | [0] | [0.011] | [0]
[0.5, 0.55] | [0] | [0.027] | [0]
[0.6, 0.8] | [1] | [0.973] | [1]
[0.7, 0.75] | [1] | [0.987] | [1]
[0.8, 0.9] | [1] | [0.999] | [1]
[0.9, 0.85] | [1] | [0.999] | [1]
[1.0, 0.95] | [1] | [1.0] | [1]

Final Loss (MSE) : 0.000173
Accuracy         : 100.0 %
```

Result

Thus the Backpropagation Artificial Neural Network was built and tested successfully on the Student Pass / Fail Data Set. The loss decreased steadily over the epochs to a final MSE of **0.000173** and the network classified the data set with an accuracy of **100.0 %**.

Problem 4: 3-Bit Odd Parity Data Set

Aim

To build an Artificial Neural Network (3-6-1 architecture) by implementing the Backpropagation algorithm in Python and to test it using the 3-Bit Odd Parity Data Set, which represents whether the number of 1s in a 3-bit input is odd.

Procedure

1. Store the input data and the expected output of the 3-Bit Odd Parity Data Set; normalise the values to the range 0 - 1 where required.
2. Fix the network structure (3 input, 6 hidden and 1 output neurons), the learning rate (0.5) and the number of epochs (20000); initialise the weights randomly and the biases with zero.
3. Forward propagation: calculate the hidden layer and output layer activations using the sigmoid function.
4. Calculate the error (expected output - actual output) and propagate it backwards to find the delta (error gradient) of the output layer and of the hidden layer.
5. Update all weights and biases using the delta rule: weight = weight + learning rate x input x delta.
6. Repeat steps 3 to 5 for all epochs while printing the loss (MSE), then test the trained network and display the predicted output.

Coding (Python)

```
# Backpropagation Neural Network - 3-Bit Odd Parity Data Set
import numpy as np

np.random.seed(1)

# Input data and expected output
X = np.array([
    [0, 0, 0],
    [0, 0, 1],
    [0, 1, 0],
    [0, 1, 1],
    [1, 0, 0],
    [1, 0, 1],
    [1, 1, 0],
    [1, 1, 1],
])
y = np.array([
    [0],
    [1],
    [1],
    [0],
    [1],
    [0],
    [0],
    [1],
])
def sigmoid(x):
```

```
    return 1 / (1 + np.exp(-x))

def sigmoid_derivative(a):              # a = sigmoid(x)
    return a * (1 - a)

# Network structure: 3 input - 6 hidden - 1 output neurons
input_n, hidden_n, output_n = 3, 6, 1
learning_rate = 0.5
epochs = 20000

# Step 1: initialise weights randomly and biases with zero
w_hidden = np.random.uniform(-1, 1, (input_n, hidden_n))
b_hidden = np.zeros((1, hidden_n))
w_output = np.random.uniform(-1, 1, (hidden_n, output_n))
b_output = np.zeros((1, output_n))

for epoch in range(1, epochs + 1):
    # Step 2: forward propagation
    hidden = sigmoid(np.dot(X, w_hidden) + b_hidden)
    output = sigmoid(np.dot(hidden, w_output) + b_output)

    # Step 3: calculate the error
    error = y - output

    # Step 4: backward propagation (delta rule)
    d_output = error * sigmoid_derivative(output)
    d_hidden = np.dot(d_output, w_output.T) * sigmoid_derivative(hidden)

    # Step 5: update weights and biases
    w_output += learning_rate * np.dot(hidden.T, d_output)
    b_output += learning_rate * np.sum(d_output, axis=0, keepdims=True)
    w_hidden += learning_rate * np.dot(X.T, d_hidden)
    b_hidden += learning_rate * np.sum(d_hidden, axis=0, keepdims=True)

    if epoch == 1 or epoch % 4000 == 0:
        print("Epoch", epoch, "  Loss (MSE) =", round(float(np.mean(error ** 2)), 6))

# Step 6: test the trained network
hidden = sigmoid(np.dot(X, w_hidden) + b_hidden)
output = sigmoid(np.dot(hidden, w_output) + b_output)
predicted = (output > 0.5).astype(int)
print("\nTesting the trained network:")
print("Input  | Actual | Output (raw)  | Predicted")
for i in range(len(X)):
    print(X[i].tolist(), "|", y[i].tolist(), "|", np.round(output[i], 3).tolist(), "|", predicted[i].tolist())
accuracy = np.mean(predicted == y) * 100
print("\nFinal Loss (MSE) :", round(float(np.mean((y - output) ** 2)), 6))
print("Accuracy         :", accuracy, "%")
```

Input

Data set (8 examples, 3 input neuron(s), 1 output neuron(s)):

A	B	C	Odd Parity
0	0	0	0
0	0	1	1
0	1	0	1
0	1	1	0
1	0	0	1

A	B	C	Odd Parity
1	0	1	0
1	1	0	0
1	1	1	1

Output

```
Epoch 1    Loss (MSE) = 0.251907
Epoch 4000   Loss (MSE) = 0.000355
Epoch 8000   Loss (MSE) = 0.000128
Epoch 12000   Loss (MSE) = 7.5e-05
Epoch 16000   Loss (MSE) = 5.2e-05
Epoch 20000   Loss (MSE) = 3.9e-05

Testing the trained network:
Input  | Actual | Output (raw)  | Predicted
[0, 0, 0] | [0] | [0.007] | [0]
[0, 0, 1] | [1] | [0.995] | [1]
[0, 1, 0] | [1] | [0.993] | [1]
[0, 1, 1] | [0] | [0.006] | [0]
[1, 0, 0] | [1] | [0.994] | [1]
[1, 0, 1] | [0] | [0.007] | [0]
[1, 1, 0] | [0] | [0.005] | [0]
[1, 1, 1] | [1] | [0.994] | [1]

Final Loss (MSE) : 3.9e-05
Accuracy         : 100.0 %
```

Result

Thus the Backpropagation Artificial Neural Network was built and tested successfully on the 3-Bit Odd Parity Data Set. The loss decreased steadily over the epochs to a final MSE of **3.9e-05** and the network classified the data set with an accuracy of **100.0 %**.

Problem 5: Half Adder Data Set

Aim

To build an Artificial Neural Network (2-4-2 architecture) by implementing the Backpropagation algorithm in Python and to test it using the Half Adder Data Set, which represents the Sum and Carry outputs of a half adder (two output neurons).

Procedure

1. Store the input data and the expected output of the Half Adder Data Set; normalise the values to the range 0 - 1 where required.
2. Fix the network structure (2 input, 4 hidden and 2 output neurons), the learning rate (0.5) and the number of epochs (10000); initialise the weights randomly and the biases with zero.
3. Forward propagation: calculate the hidden layer and output layer activations using the sigmoid function.
4. Calculate the error (expected output - actual output) and propagate it backwards to find the delta (error gradient) of the output layer and of the hidden layer.
5. Update all weights and biases using the delta rule: weight = weight + learning rate x input x delta.
6. Repeat steps 3 to 5 for all epochs while printing the loss (MSE), then test the trained network and display the predicted output.

Coding (Python)

```
# Backpropagation Neural Network - Half Adder Data Set
import numpy as np

np.random.seed(1)

# Input data and expected output
X = np.array([
    [0, 0],
    [0, 1],
    [1, 0],
    [1, 1],
])
y = np.array([
    [0, 0],
    [1, 0],
    [1, 0],
    [0, 1],
])

def sigmoid(x):
    return 1 / (1 + np.exp(-x))

def sigmoid_derivative(a):          # a = sigmoid(x)
    return a * (1 - a)

# Network structure: 2 input - 4 hidden - 2 output neurons
input_n, hidden_n, output_n = 2, 4, 2
```

```
learning_rate = 0.5
epochs = 10000

# Step 1: initialise weights randomly and biases with zero
w_hidden = np.random.uniform(-1, 1, (input_n, hidden_n))
b_hidden = np.zeros((1, hidden_n))
w_output = np.random.uniform(-1, 1, (hidden_n, output_n))
b_output = np.zeros((1, output_n))

for epoch in range(1, epochs + 1):
    # Step 2: forward propagation
    hidden = sigmoid(np.dot(X, w_hidden) + b_hidden)
    output = sigmoid(np.dot(hidden, w_output) + b_output)

    # Step 3: calculate the error
    error = y - output

    # Step 4: backward propagation (delta rule)
    d_output = error * sigmoid_derivative(output)
    d_hidden = np.dot(d_output, w_output.T) * sigmoid_derivative(hidden)

    # Step 5: update weights and biases
    w_output += learning_rate * np.dot(hidden.T, d_output)
    b_output += learning_rate * np.sum(d_output, axis=0, keepdims=True)
    w_hidden += learning_rate * np.dot(X.T, d_hidden)
    b_hidden += learning_rate * np.sum(d_hidden, axis=0, keepdims=True)

    if epoch == 1 or epoch % 2000 == 0:
        print("Epoch", epoch, "  Loss (MSE) =", round(float(np.mean(error ** 2)), 6))

# Step 6: test the trained network
hidden = sigmoid(np.dot(X, w_hidden) + b_hidden)
output = sigmoid(np.dot(hidden, w_output) + b_output)
predicted = (output > 0.5).astype(int)
print("\nTesting the trained network:")
print("Input  | Actual | Output (raw)  | Predicted")
for i in range(len(X)):
    print(X[i].tolist(), "|", y[i].tolist(), "|", np.round(output[i], 3).tolist(), "|", predicted[i].tolist())
accuracy = np.mean(predicted == y) * 100
print("\nFinal Loss (MSE) :", round(float(np.mean((y - output) ** 2)), 6))
print("Accuracy         :", accuracy, "%")
```

Input

Data set (4 examples, 2 input neuron(s), 2 output neuron(s)):

A	B	Sum	Carry
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

Output

```
Epoch 1    Loss (MSE) = 0.321097
Epoch 2000    Loss (MSE) = 0.001388
Epoch 4000    Loss (MSE) = 0.000557
Epoch 6000    Loss (MSE) = 0.000343
Epoch 8000    Loss (MSE) = 0.000247
Epoch 10000    Loss (MSE) = 0.000192

Testing the trained network:
Input  | Actual | Output (raw)  | Predicted
[0, 0] | [0, 0] | [0.013, 0.0] | [0, 0]
[0, 1] | [1, 0] | [0.984, 0.014] | [1, 0]
[1, 0] | [1, 0] | [0.987, 0.007] | [1, 0]
[1, 1] | [0, 1] | [0.019, 0.982] | [0, 1]

Final Loss (MSE) : 0.000192
Accuracy         : 100.0 %
```

Result

Thus the Backpropagation Artificial Neural Network was built and tested successfully on the Half Adder Data Set. The loss decreased steadily over the epochs to a final MSE of **0.000192** and the network classified the data set with an accuracy of **100.0 %**.